Compiler

参考编译器介绍

参考了课程组给出的sysy语言的编译器。

总体结构：编译器首先分为三部分：`src/`为源码部分，`test/`为测试部分，`tools/`为编译器开发和测试的辅助工具。

`src/`中`Compiler`为主类，在这里调用了词法分析、语法分析、生成中间代码与优化，最后输出结果。`front/`中包含了词法分析`lexer/`，语法分析`parser/`与`ast/`，前端于是可以完成词法分析语法分析，生成抽象语法树。`midend/`中完成了对于中间代码的生成。`backend/`中把中间代码翻译为目标mips代码。`error/`处理了各个阶段的错误，定义了所有的错误类型。`optimize/`中实现了各阶段需要的优化。`utils/`中有处理IO的和计算竞速复杂度的类。

接口设计：`Compiler`主类串联了各个阶段

`Frontend`：

- SetInput()：设置输入流。
- GenerateTokenList()：生成Token列表。
- GenerateAstTree()：生成AST语法树。
- GetTokenList()：获取Token列表。
- GetAstTree()：获取AST树。

`Midend`：

- GenerateSymbolTable()：生成符号表。
- GenerateIr()：生成IR中间代码。
- GetSymbolTable()：获取符号表。
- GetIrModule()：获取IR模块。

`Backend`：

- GenerateMips()：生成MIPS代码。

`Optimize`：

- Init()：初始化优化器。
- Optimize()：执行优化。

`utils`：

- IOhandler：负责输入输出的设置与打印。
- ErrorRecorder：错误记录与查询。

文件组织：`src/`为源码部分，`test/`为测试部分，`tools/`为编译器开发和测试的辅助工具。

`src/`中:

`frontend/`:前端模块，包含词法分析、语法分析、AST相关代码。子包：`lexer/`、`parser/`、`ast/`.

`midend`:`symbol/`（符号表）、`llvm/`（IR相关）、`visit/`（遍历与分析）

`backend`:BackEnd.java、MipsBuilder.java等、PeepHole.java（后端优化）

`utils`：`IOhandler.java`（输入输出）、`Setting.java`（配置）、`HandleComplexity.java`（复杂度分析）

`Compiler.java`:编译器主控类，包含主方法，负责串联各模块。

编译器总体设计

总体结构：

分为前中后端三部分，同时也设计优化部分，对全流程进行优化。前端完成词法语法分析。中端完成语义分析与中间代码生成。后端完成mips代码生成。工具来处理输入输出与错误处理还有配置管理等等。

接口设计：

前端给出GenerateTokenList()：生成Token列表，GenerateAstTree()：生成AST语法树。

中端给出GenerateIr()：生成IR中间代码。

后端给出GenerateMips()：生成MIPS代码。

优化：Init()：初始化优化器。Optimize()：执行优化。

错误处理：ErrorRecorder：错误记录与查询

文件组织：

`src/`为源码部分，`test/`为测试部分，`tools/`为编译器开发和测试的辅助工具。

`src/`中:

`frontend/`: 子包：`lexer/`、`parser/`、`ast/`，包含词法分析、语法分析、AST相关代码。

`midend`：符号表、IR相关代码

`backend`: mips汇编生成相关代码

`utils`：`IOhandler.java`（输入输出处理程序）、`Setting.java`（配置）、`HandleComplexity.java`（复杂度分析）

`Compiler.java`:编译器主控类，包含主方法，串联各模块。

词法分析设计

编码前设计

先建立Token类，作为词法分析的结果，属性包括token类型，token文本，行数。

因为token有很多类型，所以要建立一个对应关系，可以用java的给枚举类加属性的操作来实现。

还要读入文件把文件中内容转化为一个大的字符串。

然后就可以搭建有限自动机，用双指针，根据读到的内容不断推进，从而分析出各个单词。这里需要搭建多个有限自动机，然后共用一个相同的起始状态，把他们整理到一个大的有限自动机中。其中对于空白字符，先行指针跳过，然后到达非空白字符时停下，把先行赋值给后行指针。

然后得到的结果可以存在一个token的列表中，从而可以一个一个输出到文件中。

另外对于错误处理，仅需处理单个&或者|，因此包含在有限自动机的处理过程中了；但是为了方便后续编写，要建立错误类Error，并且把第几类这个信息包含进去，作为一个属性。

还要建立保留字表来确定是标识符还是保留字，这里可以建立一个映射，保留字的String与TokenType。

编码后修改设计

目前是有三个包，frontend，error，utils与不在包中的主类Compiler。

对于保留字，直接在Lexer类中写一个Map，最简单地实现保留字的映射。

对于Error类，学习教程中的给枚举类添加属性的操作，建立ErrorType类，实现了错误具体信息和类别的关系；在判断有没有错误来决定输出token还是错误的时候，根据一个全局的错误list来判断、实现。

对于文件读写设置了一个工具类FileProcess，在其中实现初始化输入输出与输入输出，还有关闭文件等等，其中要注意的是写文件还要flush。

在实现的细节上，我没有处理空白的跳过，是在for循环中自动跳过了。

可能以后要注意的点：目前错误类是静态的列表，如果多次运行要清空？

语法分析设计

编码前设计

主要要完成两部分，一个是AST的类的设计，另一部分是AST中的语法分析方法的设计。

AST的类设计：把一个推导定义为一个类，各个组成部分定义为属性。可以建立一个Node抽象类，让其他的节点全部继承Node，方便输入输出。但是具体这里它的用处还不是很清楚。

对于类中的语法分析方法，要进行预读入，然后再根据first集（这里不一定要终结符来判断，也可以用TokenType来判断）来进行判断非终结符走向。最后再用语法分析的Paser类进入开始符号来进行分析，自顶向下进行推导。

还要先建立一个TokenStream类来实现对于token流的操控，方便之后进行预读入等等操作，来判断非终结符的下面的走向。

错误类目前先用map来实现语法部分自己的错误管理。

还要记得最后输出错误的时候要进行对行号的升序排序。

对于某些文法像VarDef → Ident ['[' ConstExp ']'] | Ident ['[' ConstExp ']'] '=' InitVal，如果有文法开头为=，会不会影响互相的分析。这里也有最长匹配吗？

编码后修改设计

通过编写代码，我对于语法分析部分的了解更加充分了，通过建立AST类与定义其中的属性，相当于我规定了一棵子树，它的属性就是它的子结点。然后通过parse*的对于各种非终结符的分析方法，我对于输入的`TokenStream`从头到尾分析，并且从`CompUnit`进入，建立了一棵以`CompUnit`为根结点的语法树，可以供以后的分析使用。

在具体的设计中，我对于词法分析的错误在语法里面怎么处理这一点在之前搞错了，在词法分析中，我只是找到错误，没有帮助修正，因此这里把词法中的设计修正了，一个的& 或|，我都把他们改成两个，改成了一个正确的Token。

在debug过程中出现的一个比较容易忽略的错误就是用`!=`来判断文法中的`[]`和`{}`，就是重复0次或者一次或者多次，这里用`!=`后边的符号来判断的话，要看后边的符号是不是`;])`。因为这三者是可能丢失掉的，是错误类型，这时候再用!=来判断就会出现错误，具体还是要用`==`括号中或者分号之前的符号来判断。典型的就是`UnaryExp`。

但是在编写之后我对于这棵树怎么进行遍历来对后边的语义分析进行服务有些疑惑，有些有分支的非终结符我是用`utype`来判断，或许后边需要外部获取`utype`来判断分支走向。

语义分析

编码前设计

对于符号表，定义一个树形的符号表，虽然c语言可以用栈式符号表，但是java语言面向对象，建立一个树形的符号表也比较简单，更加高内聚低耦合。

对于Symbol中的属性，name，type，还有之后生成中间代码的相关信息（这个我暂时不知道后边要什么，先不写了）。对于Type，就用测评中的几个作为Type。

对于Symbol子类的创建，这里为了高内聚低耦合创建了VarSymbol和FuncSymbol，分别在两个子类中创建属于他们自己的属性。并且这里对于他们的构造函数，我想用ast的各种Node作为参数，重载几个构造函数，覆盖所有需要创建Symbol的情况。

这里我选择设计完符号表就要开始从根结点递归访问结点，把每个ast的结点的visit方法写在自己的类里面，其实这个和把visit方法写一个新的visit类里面是差不多的，主要是这里可能会影响之后的代码生成。

编码后修改设计

过完测试点之后，我对于语义分析（其实更多是创建符号表和错误检查）部分的理解更加深入了。这里要做完的只是这些任务，具体的深层的语义还要到生成中间代码的部分再解决。

对于设计上，我认为有几个比较困难的地方，首先是对于实参和形参的匹配的设计，这里从符号表中取出函数，得到它的形参；然后从调用函数的UnExp这里得到实参，主要的是对于类型的匹配，需要对Exp以及Exp往下推的那几个Exp类型的非终结符都判断类型，是一个递归调用的过程，从而得到Exp的类型，也就是FuncRParam的组成，由此可以判断实参的类型，可以和形参去比较来判断。

还有比较困难的地方就是对于错误判断的层次，像是`Stmt → LVal ‘=’ Exp ‘;’`中判断给常量赋值，然后在LVal里面自己判断ident是不是未定义，对于重定义我是在符号表管理中处理。

另外我加入了两个全局变量来处理错误，`curFuncSymbol`来判断是否在函数内，从而判断return语句的正确与否，这里是判断f类错误。对于g类错误，直接在函数定义的时候去判断，找Block的最后一个语句看是否是return。`inLoop`判断是否在循环类，用来辅助判断m类错误；`errorOn`判断是否再记录错误，用在预读回溯中（Stmt语法判断）；`printParser`用来判断是否输出语法成分，用途也是在预读回溯中（Stmt语法判断）。

另外这里我把错误类优化了，新加了一个管理类，通过全局静态函数来addError，这样比直接把errors列表给出更加安全。

对于我的bug，这里找到了好几个，首先是Stmt中对于`Stmt → LVal '=' Exp ';'  | [Exp] ';'`这两种的判断有问题，我之前读到ident再读到`[`或者`=`都判断为赋值，但是Exp中可以有前者，这里改为用预读Exp和回溯的方法，因为Exp也是Lval，再判断下一个符号，如果是`=`，就是Lval，否则就是Stmt，这样来判断是前者还是后者，是一种回溯的思想；还有tokenStream中用了一个没有初始化的变量来判断长度；还有对于continue和break那里笔误写错了两处；还有对于`InitVal → Exp | '{' [ Exp { ',' Exp } ] '}'`这里在构造函数笔误了；还有对于它和`ConstInitVal → ConstExp | '{' [ ConstExp { ',' ConstExp } ] '}'`，在check的时候，如果是第二种情况，要判断第一个元素是不是空。

最后耗费我最多时间的一个bug就是判断e类错误的时候，函数形参可能为null，这里不能直接`.size`，要在原函数里封装一下判断长度或者判断是否为null。